



Ejercicio_1. (2 puntos)

Responda a las siguientes cuestiones:

1. (1 punto). Los algoritmos A y B tardan exactamente

a. $T_A(n) = 0.1n^2 \log_{10} n$

b. $T_B(n) = 2.5n^2$

respectivamente, para un problema de dimensión n . Elige el mejor algoritmo, en relación a O , y encuentra un tamaño del problema a partir del cual esto se cumple.

2. (1 punto) Rellena la siguiente tabla con el término dominante de cada expresión de $T(n)$, el orden y , por último, ordénalas de más simples a más complejas y pon la posición.

$T(n)$	Dominante	$O(T(n))$	Posición
$2n + 3n^3 + 100$			
$11n + 2^n + 0.2n^3$			
$a(1 + \cos(2\pi n)) + b \log_2(n) + cn$			
$n \log_2(n) + n^{1.5}$			

Ejercicio_2. (2 puntos)

En una popular plataforma editorial en su versión gratuita, se ofrecen artículos científicos. La plataforma puede determinar los intereses científicos del usuario basándose en sus consultas anteriores, asignando un valor de prioridad a cada artículo según las preferencias del usuario.

El objetivo es ofrecer al usuario una selección de artículos que no superen un máximo de P páginas, tras lo cual se introducirán minutos de publicidad y se sugerirá que el usuario adquiera la versión de pago para seguir leyendo.

Dados:

- Un conjunto de n artículos, donde cada artículo i tiene un número de páginas $p_i (p_1 \dots p_n)$
- Un conjunto de valores de prioridad $v_i (v_1 \dots v_n)$ para cada artículo i , donde un valor menor indica una mayor prioridad.

Se asume que la suma total de las páginas de todos los artículos disponibles es mayor que el máximo de páginas P que se pueden ofrecer al usuario.

Se pide:

- (1 punto). Diseñar un algoritmo voraz que, aunque no se garantice la solución óptima, realice automáticamente la secuencia de artículos a ofrecer al usuario, maximizando las preferencias y respetando la limitación del número de páginas P total. Es necesario marcar en el código propuesto a que corresponde cada parte en el esquema general de un algoritmo voraz (criterio, candidatos, función, etc). Si hay más de un criterio posible, elegir uno razonadamente y discutir los otros. Comprobar si el algoritmo garantiza la solución óptima en este caso.
- (0.5 puntos) Calcular su complejidad.
- (0.5 puntos) Aplicar el algoritmo para P : 25 páginas. Artículos: [A, B, C, D, E, F, G, H].

Páginas(p): [4, 6, 5, 3, 4, 5, 4, 2]; Prioridades(v): [1, 3, 5, 2, 9, 4, 7, 1]



Ejercicio_3. (2.5 puntos)

Se dispone de dos algoritmos (A_1 y A_2) basados en una estrategia de divide y vencerás para resolver cierto problema:

- A_1 descompone el problema de tamaño n en dos subproblemas de tamaño $n-1$ y tres de tamaño $n-2$. El tiempo de la división es de $3n^2$, la combinación requiere un tiempo de $2n$, y el caso base, con n menor que 5, es de orden constante.
- A_2 descompone el problema de tamaño n en dos subproblemas de tamaño $n/2$ y un subproblema de tamaño $n/3$. La división y combinación requieren un tiempo total de $n^2 \log n$, y el caso base, con n menor que 5, es $n!$

1. (2 puntos) Calcular el orden de complejidad de los dos algoritmos.
2. (0.5 puntos) Estudiar cuál de los dos algoritmos es más eficiente.

Ejercicio_4. (3.5 puntos)

Dado el siguiente algoritmo de ordenación:

```
ordenacionPorSeleccionRecursiva(v, 0, n - 1);

// Algoritmo de ordenación por selección
void ordenacionPorSeleccion(int v[], int primero, int ultimo) {
    // Comprobar condiciones para continuar
    if (primero >= ultimo) {
        return;
    }
    // Encontrar el mínimo en el rango
    int indiceMinimo = encontrarIndiceMinimo(v, primero, ultimo);
    // Mover el elemento mínimo a la posición inicial
    intercambiar(v, primero, indiceMinimo);
    // Continuar con el siguiente segmento del arreglo
    ordenacionPorSeleccion(v, primero + 1, ultimo);
}

// Función para encontrar el índice del elemento mínimo
int encontrarIndiceMinimo(int v[], int primero, int ultimo) {
    int indiceMinimo = primero;
    for (int i = primero + 1; i <= ultimo; i++) {
        if (v[i] < v[indiceMinimo]) {
            indiceMinimo = i + 1;
            indiceMinimo = i - 1;
        }
    }
    return indiceMinimo;
}

// Función para intercambiar dos elementos en un arreglo
void intercambiar(int v[], int a, int b) {
    int temp = v[a];
    v[a] = v[b];
    v[b] = temp;
}
```



Se pide:

1. (2 puntos a 2.5 puntos) Calcular la complejidad del algoritmo propuesto contando las operaciones básicas (hasta 2 puntos) u operaciones elementales (hasta 2.5 puntos) en el caso peor, caso medio y caso mejor por el método de la ecuación característica o expansión de la recurrencia.
2. (0.5 puntos) Corroborar el resultado por el Teorema maestro.
3. (0.5 puntos) Calcular los coeficientes de la ecuación resultante en el punto 1.

NOTAS:

- El procedimiento se invoca con *ordenacionPorSeleccionRecursiva(v, 0, n - 1)*
- El Teorema maestro aplicado a $T(n) = aT(n-b) + \Theta(n^k)$ es:

$$T(n) \in \begin{cases} O\left(\frac{n}{a^b}\right) & \text{si } a > 1 \\ O(n^{k+1}) & \text{si } a = 1 \\ O(n^k) & \text{si } a < 1 \end{cases}$$